arXiv:2606.21454v1 [cs.AR] 19 Jun 2026

COMPOSE: Static Timing-driven Composable Reconfigurable Architecture for Accelerating Recurrence-Bound Loops

Rohan Juneja
rohan@comp.nus.edu.sg
School of Computing
National University of Singapore

Vishruti Ranjan
vishruti@comp.nus.edu.sg
School of Computing
National University of Singapore

Rakshith Harish
Rakshith_Harish@a-star.edu.sg
Institute of Microelectronics
A*STAR

Li-Shiuan Peh
peh@comp.nus.edu.sg
School of Computing
National University of Singapore

Abstract

Coarse-Grained Reconfigurable Architectures (CGRAs) provide a spatially programmable substrate well suited for accelerating compute-intensive workloads with abundant parallelism. However, traditional CGRA execution models rely on rigid, fixed-size processing elements (PEs) that are statically bound to individual operations, which forces inter-iteration dependencies to be resolved through serialized scheduling. This limits throughput and reduces parallelism across loop iterations. Moreover, static execution schedules often fail to exploit available timing slack between operations, leading to resource underutilization and increased latency. The frequent registering of intermediate results further exacerbates pressure on register files and local memories, introducing data movement overheads that reduce energy efficiency, particularly in power or memory constrained environments.

To address these challenges, we introduce *COMPOSE*, a composable CGRA architecture that enables dynamic formation of PEs at compile time guided by static timing information. By spatially fusing operations across loop iterations and selectively utilizing slack, *COMPOSE* resolves inter-iteration dependencies that limit throughput and enables low latency execution by reducing slack wastage. Additionally, the architecture reduces register file pressure by deferring output registration when intermediate values remain locally consumable, which significantly lowers redundant memory traffic. Across a diverse set of workloads, *COMPOSE* on average delivers 1.6x performance improvement and 2.9x EDP reduction over state-of-the-art (SOTA), at minimal area and power overheads.

Keywords

Coarse-Grained Reconfigurable Architectures (CGRAs), Static Timing Analysis (STA), Critical-Path-Delay

1 Introduction

CGRA Design Space. Coarse-grained reconfigurable architectures (CGRAs) bridge the gap between fine-grained FPGA lookup table fabrics and fixed-function ASIC accelerators by providing a spatially programmable array of processing elements (PEs) connected by a reconfigurable interconnect. Both the industry [1, 17, 18, 27] and academia [6, 10, 20, 25, 26, 29] have built many variants that differ in array size, interconnect structure, and mix of compute and memory tiles, all with the goal of mapping compute kernels onto a regular fabric for high throughput and good energy efficiency.

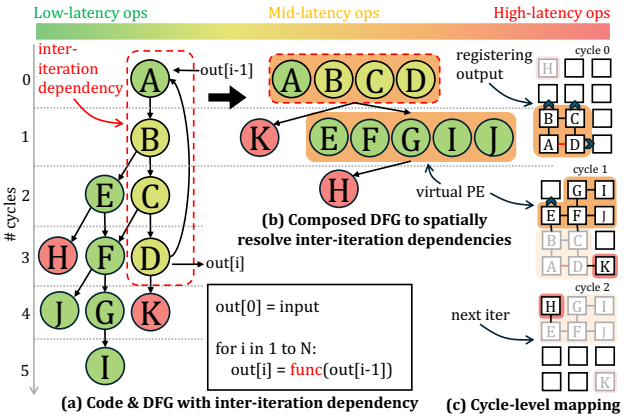


Figure 1: Timing-aware spatial composition in *COMPOSE*. Fusing low-latency operations into virtual PEs absorbs unexploited timing slack, compressing the DFG depth and resolving inter-iteration dependencies without violating the global clock period.

These architectures are mapped with a software toolchain that starts from a kernel and builds a dataflow graph (DFG) where nodes are operations and edges are data dependencies. The mapper targets throughput via the initiation interval (II), the number of cycles between the start of consecutive loop iterations. Smaller II means higher steady-state throughput. It then selects an unrolling factor to expose parallelism, places operations on PEs, and routes dependencies through the interconnect under resource and connectivity constraints. Finally, the toolchain builds a schedule, and outputs the configuration for PEs and switches in the interconnect.

Revisiting PE Boundaries in CGRAs. Prior CGRA research lacks a unified abstraction for the Processing Element (PE). While some define a PE as a fixed-function unit and others as a complete ALU-based compute node, we adopt a more precise definition: the PE is the atomic scheduling unit to which exactly one operation is mapped per clock cycle. Under this definition, conventional CGRA compilers face significant inefficiencies when mapping complex or composite instructions. Standard flows decompose high-level operations into primitive instructions, each partitioned by a rigid cycle boundary that necessitates a write-back and subsequent read

from a local register file. These boundaries waste timing slack, as the global clock period is governed by the critical path of the worst-case operation. For inter-iteration data dependencies, this forces unnecessary serialization, thereby increasing execution latency and degrading effective throughput.

This inefficiency is further exacerbated in power-constrained domains where architectures are operated at reduced frequencies. In edge devices, for instance, designers often down-clock high-frequency fabrics to minimize dynamic power, resulting in substantial unexploited timing slack.

Proposed Composable PEs. These inefficiencies arise because conventional CGRA compilation flows remain architecturally agnostic to fine-grained circuit-level timing, assigning a uniform clock cycle budget to all operations regardless of their physical delay. In practice, critical-path-delays exhibit significant variance; while high-latency units define the global clock period, low- and mid-latency operations execute much faster, leaving substantial unexploited timing slack. As illustrated in Fig. 1(a), the mapping of our motivational DFG onto a conventional fabric assigns each operation to a discrete cycle (spanning 6 cycles in total), ignoring the potential to exploit this slack.

To address this, we introduce *COMPOSE*, a timing-aware framework that leverages static timing analysis to dynamically establish "virtual PE" boundaries constrained by the target clock period and physical resource availability. As shown in Fig. 1(b) and (c), *COMPOSE* harvests the slack inherent in high-speed operations to spatially chain dependent producer-consumer pairs (e.g., grouping nodes A, B, C, and D) within a single clock cycle, without violating the timing constraints set by the slowest path. This spatial composition is realized through a low-latency hardware bypassing mechanism that enables direct combinational chaining across PEs, effectively bypassing the standard clocked register boundaries (detailed in Section 3) and only registering the output at the boundary of the virtual PE.

This provides 3 key advantages: (1) it enables sub-cycle recurrence resolution by composing PEs such that critical inter-iteration dependencies complete within fewer cycles (e.g., the loop-carried dependency from ($D_{out[i]}$ to $A_{out[i-1]}$), (2) it collapses the DFG depth by fusing combinational chains to minimize input-to-output latency (reducing execution from 6 cycles to 3), and (3) it reduces register-file pressure and dynamic power by achieving equivalent or superior performance at a reduced clock frequency, effectively trading unexploited timing slack for significant energy savings.

Positioning relative to prior work. Table 1 positions *COMPOSE* along three axes: the design stage at which PE composition is decided (fabrication vs. compilation), whether mapping is slack-aware, and whether it explicitly handles inter-iteration dependencies. Fabrication-time approaches enrich the PE itself. This design-time specialization lineage traces back to configurable computation accelerators (CCAs) for instruction-set customization. Clark et al. [11–13, 22, 50] introduce CCAs that execute selected dataflow subgraphs on a configurable array of function units, with follow-on work exploring transparent accelerator design spaces. EGRA [2–4] brings this direction into the CGRA setting by mapping larger subgraphs onto reconfigurable multi-ALU clusters to improve compute density, while APEX [30] merges frequent subgraphs into synthesized PE RTL, using timing analysis to set the pipeline depth needed

Related Works	PE composition decided at	Compilation Slack-aware?	Inter-iter. deps. Aware?
EGRA [2, 4]	Fabrication	✗	✗
APEX [30]	Fabrication	✓	✗
Slack-aware sched. [5]	Compilation	✓	✗
CGRA-Express [36]	Compilation	✓	✗
Cascade [31]	Compilation	✓	✗
Recurrence-aware sched. [35]	✗	✗	✓
UE-CGRA [47]	✗	✗	✓
COMPOSE	**Compilation**	✓	✓

Table 1: Comparison with prior CGRA composition and scheduling approaches. *COMPOSE* is the only compilation-time approach that is both slack-aware and explicitly accounts for inter-iteration dependencies.

to meet an aggressive frequency, effective for machine-learning and image-processing kernels with frequently occurring subgraphs. Because both lock operation boundaries at fabrication, composition is set once for the fabric and reused across kernels rather than tuned per kernel. Compilation-time approaches move this flexibility into the mapper: slack-aware scheduling [5] folds per-operation timing variation into modulo scheduling, and CGRA-Express [36] fuses operations through a bypass network. Cascade [31] instead pipelines interconnect paths post-PnR to raise frequency, which aids feed-forward kernels but cannot shorten loop-carried recurrences. Both exploit sub-cycle slack and are closest to *COMPOSE*, but restrict fusion to neighboring PEs, whereas *COMPOSE* forms VPEs across multiple hops by folding interconnect delay into the timing budget; They are also agnostic to loop-carried paths. A separate line of work targets inter-iteration dependencies directly: recurrence-aware scheduling [35] groups operations along recurrence cycles atop edge-centric modulo scheduling (EMS) to improve schedule quality, and UE-CGRA [47] accelerates loop-carried paths via PE-level DVFS but incurs significant area and power overhead (>30%), yet neither harvests circuit-level slack. *COMPOSE* is the only compilation-time approach that is both slack-aware and explicitly recurrence-aware: it uses STA-derived delays to form virtual-PE boundaries around producer-consumer chains, particularly inter-iteration dependencies, collapsing them into single registered stages to reclaim wasted slack and resolve the inter-iteration dependencies that bound throughput.

Our contributions include:

- a methodology and detailed characterization of combinational timing paths derived from a silicon-proven CGRA to establish a high-fidelity baseline for circuit-aware mapping.
- a timing-driven, recurrence-aware framework, *COMPOSE* that uses STA to form virtual-PE boundaries: it harvests sub-cycle slack and co-locates inter-iteration dependencies within fewer registered stages to improve performance and lower input-to-output latency, generating a Pareto-optimal frontier across operating frequencies.
- integrating the *COMPOSE* flow into an open-source CGRA infrastructure and demonstrating its efficacy across diverse architectural configurations and real-world kernels, achieving significant performance and energy gains.

Results: Experimental results show that *COMPOSE* achieves, on average, 2.3x lower normalized cycle count, 6.3x better energy-delay product (EDP), and 2x lower input-to-output latency, while demonstrating that peak performance and efficiency can be achieved at much lower operating frequencies by composing PEs. *COMPOSE* further improves resource utilization by 2x and reduces register pressure by 2.2x compared to SOTA CGRAs.

2 Background and Motivation

This section first provides background on FO4 delay, before diving into a case study of a CGRA chip fabricated on a commercial 12nm node. We perform static timing analysis (STA) of the timing paths on 12nm and 40nm processes, translated into FO4 delays as a technology-independent metric, across six CGRA kernels. Our case study reveals substantial potential for composing primitives within a single cycle, motivating *COMPOSE*.

2.1 FO4 delay: A technology-independent metric for timing

The timing closure of modern high-performance spatial architectures depends on rigorous Static Timing Analysis (STA) to guarantee functional correctness across all process, voltage, and temperature (PVT) corners [7]. As illustrated in Fig. 3, the non-linear scaling of path delays across successive technology nodes highlights the challenge of maintaining a unified timing abstraction. When delays are expressed in absolute temporal units (ps), the architectural model becomes tied to a specific process, complicating the development of hardware-aware software that must remain portable across technology generations. Consequently, achieving high operating frequencies in advanced technology nodes requires Statistical STA (SSTA) to mitigate the effects of IR drop and crosstalk-induced jitter, which are exacerbated by the reduced noise margins of low-voltage (Vdd) operation [8].

To ensure a technology-independent evaluation of architectural merit, the Fan-out-of-4 (FO4) delay is utilized as the canonical metric for normalizing logic depth. Formally defined as the propagation delay of a CMOS inverter driving four identical inverters, FO4 encapsulates the intrinsic switching speed of a specific process [45]. While the absolute delay of an FO4 unit scales down — from approximately 30ps at 65nm to sub-10ps in 7nm FinFET [23, 44] — it serves as a normalized "yardstick" that remains consistent across different fabrication technologies. By expressing the critical path in terms of FO4 units ($T_{clk}/FO4$), the logical depth of an operator remains relatively comparable in FO4 terms across technology nodes, as illustrated in Fig. 3. Thus, FO4 provides a stable baseline for identifying whether performance bottlenecks arise from architectural complexity or physical scaling limits, enabling effective cross-node benchmarking and hardware-aware optimizations. For VPE formation, *COMPOSE* uses absolute post-layout delays for the target node. **FO4 has two roles: it shows the delay spread is structural, with the 12nm and 40nm series tracking within 13% in FO4 terms (Fig. 3), and it provides default cost models when retargeting before signed-off STA exists.**

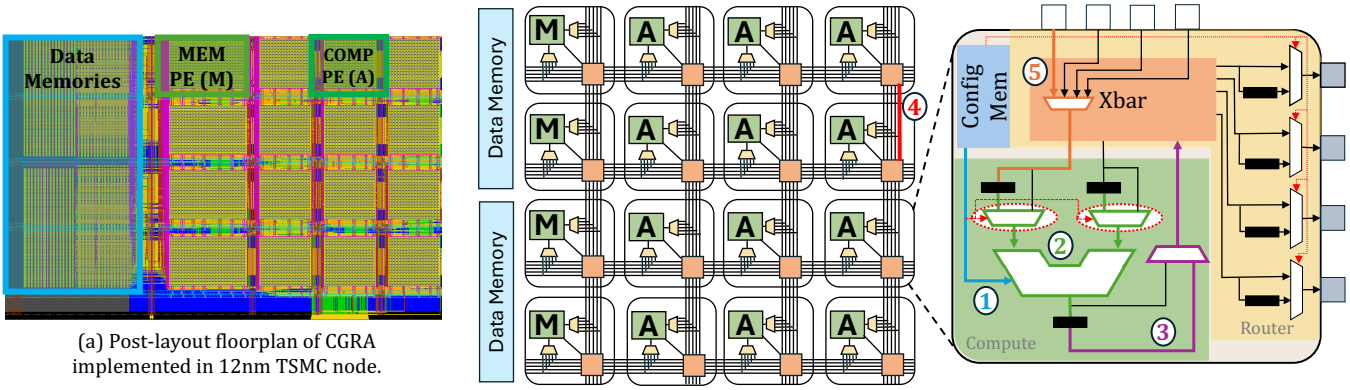
2.2 Motivating case study

Our analysis is based on a silicon-proven, energy-efficient CGRA architecture implemented in a 12nm FinFET commercial process that achieves a peak energy efficiency of 1.31 TOPS/W while meeting a target operating frequency of 1 GHz. As illustrated in Fig. 2(a), the floorplan shows a single 4×4 cluster of PEs. To provide sufficient memory bandwidth for data-intensive kernels, the four edge PEs of each cluster are specialized Memory-capable PEs (MEM) equipped with dedicated Load-Store Units (LSUs) which interface with a shared 4-port data memory. The remaining twelve internal tiles are Compute-only PEs optimized for high-throughput arithmetic kernels. This clustered, heterogeneous configuration reflects industry CGRAs like SambaNova SN10 [37] and Efficient Electron E1 [15], providing a representative baseline.

Synthesis and Physical Implementation: The architecture was implemented and taped out in the TSMC 12nm FinFET process. Logic synthesis was performed using Cadence Genus with a multi threshold-voltage CMOS (MTCMOS) strategy. Specifically, Ultra-Low-Vt (uLVT) cells were strategically prioritized for the critical paths within the PE mesh to maximize operating frequency. The physical floorplanning and structured placement were executed in Cadence Innovus via a custom Tcl-based placement script. As illustrated in the post-layout floorplan (Fig. 2(a)), this script enforces strict spatial regularity by defining a $0.001um$ coordinate grid, ensuring the fabric maintains the desired architectural symmetry. Notably, the MEM PEs are allocated a much larger silicon footprint, ~1.45X the area of a standard compute PE, to accommodate the dedicated LSU logic and high-bandwidth memory port interfaces. Final routing was performed using Innovus NanoRoute, leveraging the TSMC 12nm middle metal layers between M4-M7 for low-resistance global interconnects between the four clusters, while top-metal layers M7-M9 were used for power delivery.

Static Timing Analysis and FO4 Normalization: Final timing sign-off was conducted using Cadence Tempus with post-routing parasitics (RC) input from the Standard Parasitic Exchange Format (SPEF) file. As illustrated in Fig. 2(b), the cycle time is determined by the longest PE-to-PE critical path, which we decompose into five timing segments for fine-grained characterization: ① configuration memory to ALU input selection, ② pure combinational ALU computation, ③ ALU output to the local routing crossbar, ④ router-to-router propagation along the programmed path, and ⑤ the final hop into the destination PE, accounting for clock skew and output registering. Delays for the five paths were reported for TT corner with sign-off settings.

To establish a process-independent baseline, we characterize an inverter chain in the TSMC 12nm library, where each stage drives a capacitive load of four identical inverters. The resulting delay 3.24 ps defines one FO4 unit. By mapping the absolute delays from our five-arc STA across multiple PVT corners to this FO4 constant, we generate a normalized timing graph. To validate the consistency of this abstraction across disparate process nodes, we also performed the same physical implementation and STA flow for a 40nm UMC technology node, where one FO4 corresponds to 10.9 ps. As shown in Fig. 3, the resulting data captures the delays for various operations and interconnect paths across 12nm, 40nm, and the corresponding FO4 metrics.



(a) Post-layout floorplan of CGRA implemented in 12nm TSMC node.

(b) 4x4 CGRA, with additional muxes for bypass paths. Edge PEs support both memory & compute operations. Annotated arcs (1–5) correspond to segments used for granular STA.

Figure 2: Physical implementation and timing paths of a silicon-proven CGRA chip with support for *COMPOSE*.

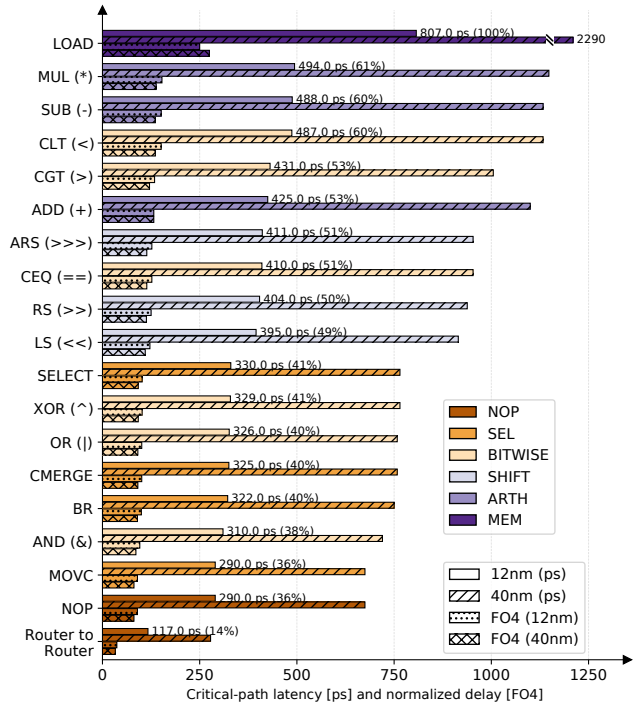


Figure 3: Critical-path delays in 12nm and 40nm technologies. Absolute delays (ps) are shown alongside technology-normalized FO4 counts, illustrating that operator logical depth remains largely unchanged across technology nodes.

This normalization highlights the significant timing variance between different operation classes. Even within an ALU, many operations are effectively wiring or selection operations, such as *MOVC*, *SEXT*, *SELECT*, and *CMERGE* (described in Table 1), which perform no computation but are necessary for routing data across fabric. Their delay is dominated by small multiplexers and short local wires, leaving substantial slack. Next are single-level bitwise and predicate operations such as *OR*, *AND*, *XOR*, *CMP*, *CGT*, and *CLT*, which add one gate level and modest flag logic, yielding slightly higher but still low delay. Shifts such as *RS*, *ARS*, and *LS* sit in the middle because a barrel shifter introduces wider mux trees and more wiring. Arithmetic primitives are the slowest by comparison: wide *ADD* and *SUB* are limited by carry propagation, while *MUL* is typically the longest path and often sets the ALU's critical-path. Memory operations stand apart as the most significant bottleneck; due to the cumulative delay of memory macros, arbitration logic, and LSU overhead, these operations are so long they typically require two clock cycles to complete. Fig. 3 shows the variation in critical-path-delay for all these paths. **This spread explains why a uniform cycle budget wastes slack on most nodes and motivates composing fast primitives within a single cycle.**

2.3 Analysis of applications

Fig. 4 characterizes six kernels with their source code and dataflow graphs. Each node is colored by its intrinsic critical-path delay based on the aforementioned post-synthesis timing data. Green nodes represent low-latency operations, while red nodes signify high-delay primitives. Dashed edges denote loop-carried dependencies, which establish the fundamental recurrence boundary. Slack is the temporal difference between the global clock period and a node's specific execution delay. Consequently, greener nodes represent a significant underutilization of the available cycle budget.

Our analysis reveals two primary inefficiencies in standard CGRA mapping. First, in kernels with true inter-iteration data dependencies, such as the pointer-chasing in *llist* or the recursive stages in *fft*, the DFG contains a long loop-carried path that dictates the initiation interval (*II*). Because a new iteration can only begin every *II* cycles, the throughput is strictly bottlenecked by the recurrence delay, regardless of the timing of individual nodes. Second, bitwise-heavy kernels such as *viterbi*, *crc32*, and *sha256* exhibit a "slack abundance." Most nodes are green, yet the global clock frequency is penalized by a small minority of slow "red" nodes. This mismatch results in a large amount of unused timing slack in every cycle. Furthermore, the standard practice of writing every intermediate result to a PE-boundary register before the next consumer executes introduces mandatory register file energy overheads that do not contribute to functional throughput. Regular kernels such as *spmspm* and *sddmm* also exhibit recurrence edges in their raw

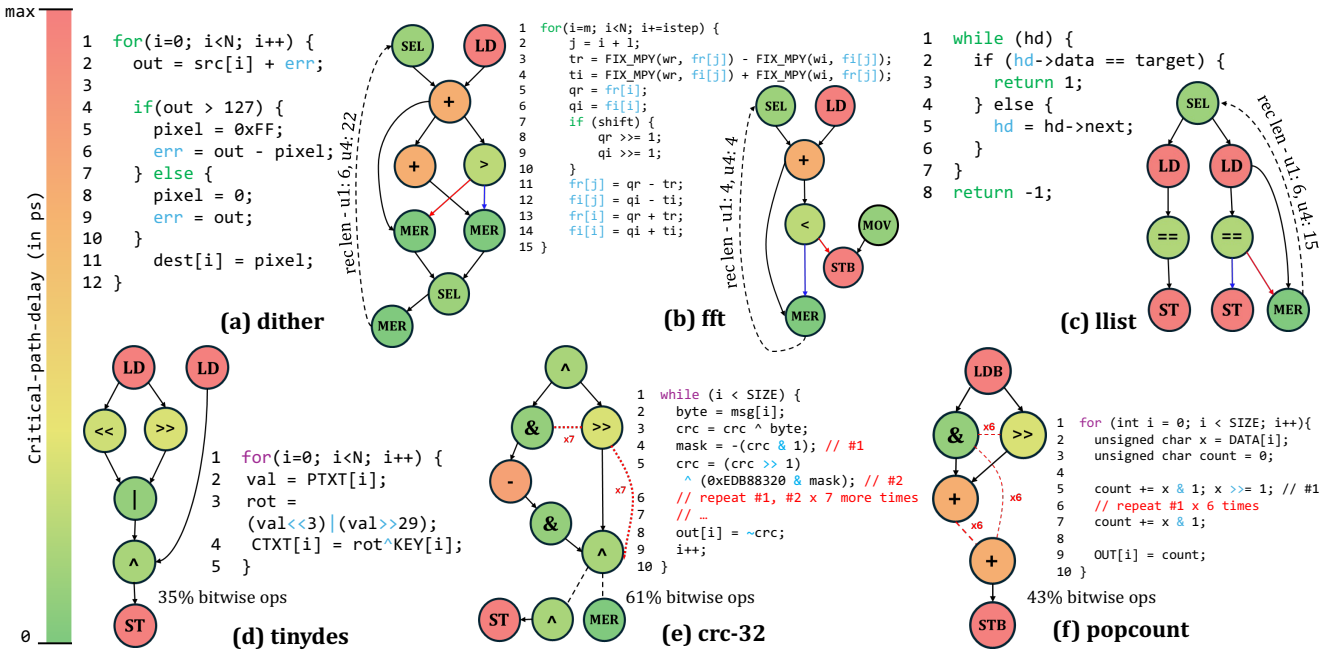


Figure 4: Six kernels with code and dataflow graphs. Nodes are colored by per-node critical-path-delay (scale at left: green low, red high); the dashed edge marks the loop-carried dependence. *Dither*, *fft*, and *llist* show long back edges (recurrence lengths written for unroll 1 and 4) that bind the initiation interval, whereas *tinydes*, *crc-32*, and *popcount* are dominated by bitwise and select nodes, leaving substantial slack.

DFGs even though their iterations are independent. However, these are typically induction variable dependencies that can be decoupled from the core computation, modern literature and commercial CGRAs [1, 34, 48] mitigate this by offloading such dependencies to specialized hardware loop-counters or AGUs to maintain high throughput.

Benchmark	Memory (%)	ALU (%)	Bitwise (%)	Wiring (%)
dither	17.9	17.8	14.3	50
llist	36.8	10.5	21.1	31.6
fft	25.4	29.9	34.3	10.4
susan	24.2	30.3	12.2	33.3
bfs	32.3	17.7	14.7	35.3
viterbi	23.7	15.8	15.8	44.7
tinydes	26.1	17.4	34.8	21.7
popcount	14.3	28.6	42.9	14.2
aes	18.7	12.3	59	10
crc32	8.2	18	60.7	13.1
gemm	38.4	27	15.4	19.2
conv2d	25.6	43.6	18	12.8
spmspm	46.4	21.4	14.3	17.9
sddmm	42.9	28.6	10.7	17.8

Table 2: Breakdown of operation types across the kernels

As detailed in Table 2, operation distribution exhibits high variance across application domains, demonstrating the inefficiency of a one-size-fits-all cycle budget. Cryptographic and mathematical kernels (e.g., crc32, popcount) are dominated by low-latency bitwise logic, which comprises up to 60.6% of the DFG. Workloads bottlenecked by complex loop iteration dependencies and irregular control flow (e.g., dither, viterbi, bfs) allocate a substantial fraction of nodes to wiring and selection logic, ranging from 31% to 50%. Regular dense workloads (e.g., gemm, conv2d, spmspm) primarily utilize high-latency ALU (up to 43.6%) and memory operations (up to 46.4%). Despite this varied operation mix, memory nodes consistently dictate the global critical path [10] even in kernels where they are numerically infrequent. This timing bottleneck originates from the cumulative delay of memory macros, memory-tile arbitration logic, and Load-Store Unit (LSU) overhead. Furthermore, in advanced process nodes, the long-range interconnect required to route data from the memory periphery across the PE array introduces high RC parasitics that frequently exceed the gate delay of the functional units. Consequently, these factors constrain the global f_{max}, generating significant unexploited timing slack within the highly prevalent bitwise and wiring operations mapped to the spatial fabric.

To reclaim this unexploited slack, we leverage the inherent configurability of spatial architectures. Unlike specialized accelerators where the dataflow is often hard-wired, CGRAs offers a unique opportunity for tight hardware-software co-design. As CGRAs are generalized configurable architectures, the compiler plays a central role in orchestrating execution. This requires hardware and software teams to work hand-in-hand to ensure spatial mappings remain physically viable. Such a tight coupling allows the mapper to access chip timing results that are typically opaque to software

teams, effectively establishing timing as a first-class constraint within the hardware-software interface.

Prior work has optimized the first two families well: APEX [30] accelerates machine-learning and image-processing kernels by mining frequent subgraphs and synthesizing high-frequency specialized PEs, while slack-aware scheduling [5] and CGRA-Express [36] exploit the abundant slack in bitwise-heavy kernels through compile-time operation fusion. We include these workloads to demonstrate the framework's versatility across diverse operation mixes, but *COMPOSE* primarily targets kernels whose throughput is bound by inter-iteration dependencies, where neither higher frequency nor local fusion resolves the serialized loop-carried path.

3 Design of *COMPOSE*

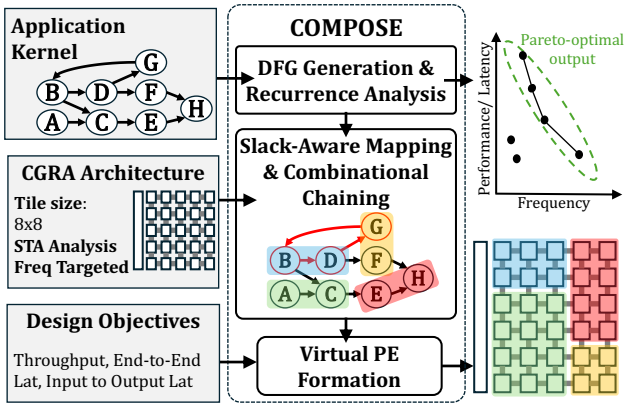


Figure 5: *COMPOSE* Framework

The *COMPOSE* framework unifies circuit-level timing insights into a hardware-driven compilation strategy to optimize mapping for CGRAs. As illustrated in Fig. 5, *COMPOSE* takes application kernels and architectural specifications, specifically post-layout STA data, as primary inputs. The framework consists of three key stages:

DFG Generation & Recurrence Analysis: The framework initially transforms high-level kernel code into a Dataflow Graph (DFG) where nodes represent primitive operations and edges denote data-dependency. A critical sub-phase involves Recurrence Edge Analysis to identify loop-carried dependencies. These cycles fundamentally bound the Recurrence Minimum Initiation Interval (RecMII), creating a hard ceiling on steady-state throughput.

Slack-Aware Mapping & Combinational Chaining: In this phase, the framework executes a slack harvesting strategy that integrates the total combinational delay of the data routing path with internal operation latencies. Rather than treating nodes as isolated units, the mapper evaluates the complete propagation delay, accounting for multi-hop interconnect latency and switching overheads required to reach a destination. It greedily chains producer-to-consumer operations such that their cumulative path delay, inclusive of all routing and switching overhead, is bounded by a target clock period (T_{clk}).

Virtual PE Formation: At compile time, *COMPOSE* defines Virtual PE (VPE) boundaries that decouple logical operation mapping

Algorithm 1: DFG Generation and Recurrence Analysis

Input: Loop body L, Instruction set I, Control Flow Graph *CFG*
Output: DFG $G(V, E)$, Recurrence map *RecII*

$V \leftarrow \emptyset$; $E \leftarrow \emptyset$; $RecII \leftarrow \emptyset$
▷ Step 1: Identify CFG back-edges & build forward-reachability sets
BackEdges ← FindBackEdges(*CFG*)
for *each basic block B in CFG* **do**
  FwdReach[B] ← DFS(B, *CFG* \ *BackEdges*) ▷ BBs reachable without crossing back-edges
▷ Step 2: Create DFG nodes with post-layout timing
for *each instruction* $i \in I$ **do**
  $v \leftarrow$ CreateNode(i)
  Annotate(v, STA(i)) ▷ attach post-layout delay $\delta(v)$
  $V \leftarrow V \cup \{v\}$
▷ Step 3: Edge construction and recurrence identification
for *each* $v \in V$ **do**
  for *each operand of* v *produced by* $u \in V$ **do**
    $E \leftarrow E \cup \{(u, v)\}$
    if $BB(v) \notin FwdReach[BB(u)]$ **then**
      $RecII[(u, v)] \leftarrow 1$ ▷ loop-carried: destination not forward-reachable from source
    else
      $RecII[(u, v)] \leftarrow 0$ ▷ intra-iteration edge
return $G(V, E)$, *RecII*

from the fixed boundaries of individual PEs. By spatially fusing subgraphs into single-cycle composites, the framework (1) reduces end-to-end execution latency, (2) improves throughput in dependency-bound kernels by collapsing multi-cycle recurrence paths into fewer cycles, and (3) minimizes data movement overhead by deferring output registration for locally consumable intermediate values, significantly reducing register file traffic to enhance energy efficiency.

The framework generates a Pareto-optimal frontier of design points, generating multiple schedules that explore the trade-offs between throughput, latency, and energy efficiency. This enables the user to select the specific schedule that best aligns with their design objectives, whether maximizing performance for compute-intensive workloads or minimizing power for energy-constrained environments.

3.1 DFG Generation & Recurrence Analysis

Loop-carried dependencies (recurrences) impose a fundamental throughput bottleneck on CGRA execution: if a recurrence path spans d pipeline stages, the architecture cannot initiate a new iteration until d cycles have elapsed, setting $RecMII \geq d$ [39]. By identifying recurrence edges at the DFG level and co-locating their endpoints within a single VPE, *COMPOSE* collapses multi-stage recurrence paths into few registered cycles, directly reducing *RecMII* and improving steady-state throughput.

However, this recurrence-first strategy introduces a latency trade-off. As illustrated in Fig. 6, prioritizing recurrence co-location can force a new VPE to be instantiated at Node 2 despite underutilized slack in VPE 0, solely to ensure the recurrence completes within a single cycle wherever possible. While this maximizes throughput for dependency-bound loops, it fragments non-critical operations across additional pipeline stages, increasing end-to-end latency.

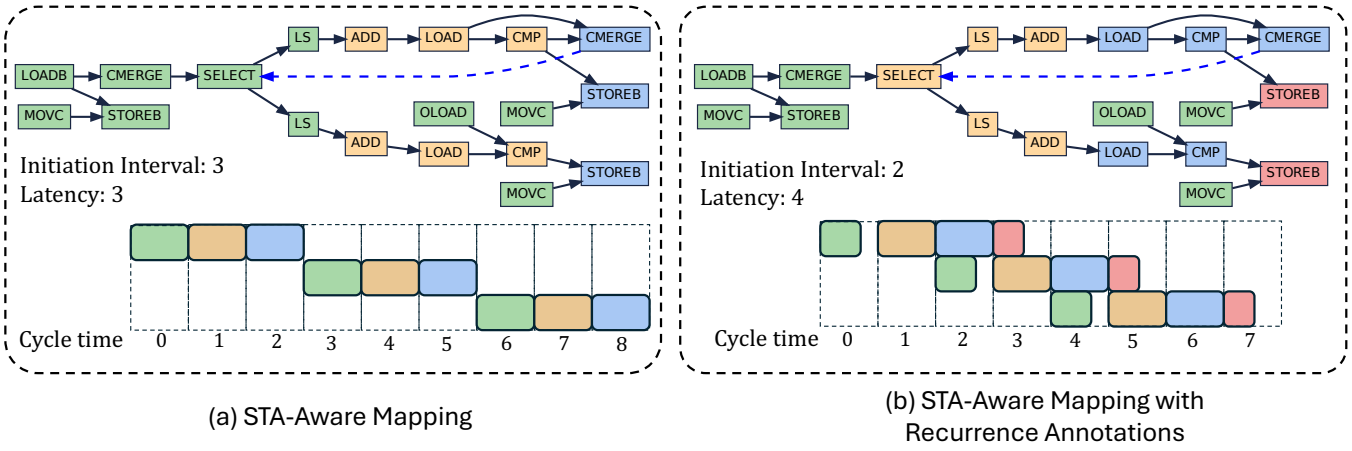


Figure 6: STA-Aware Mapping with and without Recurrence Co-location, where each color represents a distinct composite. (a) Without recurrence awareness, operations span 3 pipeline stages, giving II = 3. (b) Co-locating recurrence-dependent operations within a single cycle reduces II to 2, but fragments non-critical operations across an extra pipeline stage, increasing input-to-output latency from 3 to 4.

COMPOSE exposes this tradeoff by generating multiple mapping points across the throughput-latency Pareto frontier, allowing the user to select the schedule that best matches their design objective.

The first stage of the *COMPOSE* compiler transforms a loop body into a DFG annotated with post-layout timing and recurrence information, as presented in Algorithm 1. The algorithm begins by identifying all control-flow back-edges in the loop's CFG using a standard depth-first traversal (Step 1). For each basic block *BB*, a forward-reachability set $FwdReach[BB]$ is constructed by performing a DFS over the CFG while excluding back-edges. This set captures all basic blocks reachable from *BB* within a single loop iteration, and serves as the basis for distinguishing intra-iteration edges from loop-carried (recurrence) edges.

Next, each instruction is instantiated as a DFG node and annotated with its post-layout operation latency $\delta(v)$, obtained from gate-level STA (Step 2). During edge construction (Step 3), the algorithm classifies each data dependence: if the destination basic block $\mathrm{BB}(v)$ is not in $FwdReach[\mathrm{BB}(u)]$, the edge (u, v) is loop-carried and marked $RecII[(u, v)] = 1$; otherwise it is an intra-iteration edge with $RecII[(u, v)] = 0$. This forward-reachability test cleanly handles both explicit PHI-node recurrences and implicit topological back-edges without requiring pattern-specific detection logic.

The recurrence map *RecII* directly informs the downstream mapper in two ways. First, recurrence edges define hard constraints on the initiation interval: operations connected by recurrence edges must complete within a bounded number of cycles, establishing the *RecMII* lower bound on *II*. Second, the mapper uses recurrence edges to co-locate producer-consumer pairs into the same Virtual PE, ensuring that loop-carried paths complete within a single registered stage wherever possible.

A fundamental challenge is determining whether to partition the DFG into single-cycle composites prior to or during the physical mapping phase. While ahead-of-time partitioning based on critical-path delay might simplify the mapper flow, it fails to account for the stochastic nature of spatial resource contention. Specifically, partitioning at the DFG level cannot guarantee hardware feasibility within a single cycle because physical resource constraints—primarily routing congestion—are not visible until the mapping is done. This approach is essential for architectures with single-cycle, multi-hop interconnects, where routing overhead is highly variable [24] and sensitive to the specific spatial placement of PEs.

To address this, *COMPOSE* defers partitioning to the mapping stage. If the mapper encounters a resource bottleneck that prevents a node from being placed within the current timing-valid composite, it automatically registers the output and initiates a new VPE. By deferring composition until physical placement, the framework accounts for actual routing delays—including multi-hop interconnect latency d_{hop} from post-layout PnR data—ensuring single-cycle chains are practically realizable within the global T_{clk} constraint.

3.2 Slack-Aware Mapping & Combinational Chaining

Algorithm 2 outlines the mapper algorithm, which begins by performing a forward STA over the acyclic subgraph of the DFG—all edges excluding recurrence edges identified in Section 3.1 (Phase 1 of Alg. 2). For each node v, the cumulative arrival time $arr[v]$ is computed as the maximum arrival among its forward predecessors plus its own post-layout operation latency $\delta(v)$, sourced from gate-level STA data. Recurrence edges are skipped in this traversal since their values arrive from the previous loop iteration via a latch.

Simultaneously, the mapper establishes recurrence groups using a Union-Find structure: all nodes connected by recurrence edges are unified into equivalence classes. Nodes are then sorted by arrival time in ASAP order to establish the mapping priority. Phase 2 computes the initial *II* by evaluating each recurrence group *C*: the minimum number of clock cycles required to execute all operations in *C* is $\lceil \sum_{v \in C} \delta(v) / T_{\mathrm{clk}} \rceil$, and *RecMII* is the maximum across all groups. This provides a tight lower bound on the initiation interval before any physical mapping is attempted.

Algorithm 2: Slack-Aware Virtual PE Mapping

Input: $G(V, E)$, *RecII*: Annotated DFG from Alg. 1. *RecII*$[(u,v)] \in \{0,1\}$: 1 for recurrence, 0 for intra-iteration. $\mathcal{A}$: CGRA ($X \times Y$). δ: Post-layout op. latencies (ps). d_{hop}: Per-hop interconnect delay (ps, from PnR). T_{clk}: Clock period (ps).
Output: $\mathcal{M}$: Node-to-PE mapping. II: Initiation interval. L: Pipeline latency (number of VPE stages).

▷ Phase 1: STA and recurrence analysis — establish mapping order and recurrence groups
for *each* $v \in V$ *in topological order over forward edges (RecII*$[(u,v)]=0$*)* **do**
  $arr[v] \leftarrow \delta(v) + \max_{(u,v)\in E,\, RecII[(u,v)]=0} arr[u]$ ▷ cumulative delay from sources; recurrence edges skipped
Initialize Union-Find $\mathcal{U}$ over V
for *each* $(u,v) \in E$ *where RecII*$[(u,v)]=1$ **do** $\mathcal{U}$.Unite(u,v) ▷ group recurrence-connected nodes
Sort V by $arr[v]$ in ASAP order
▷ Phase 2: Compute initial II from recurrence structure
$RecMII \leftarrow 0$
for *each recurrence group* $C \in \mathcal{U}$ **do** $RecMII \leftarrow \max(RecMII, \lceil \sum_{v\in C} \delta(v)/T_{\text{clk}} \rceil)$ ▷ min cycles for recurrence group
$II \leftarrow \max(RecMII, 1)$
▷ Phase 3: Incremental VPE formation during mapping
while $II \le II_{\max}$ **do**
  for $r = 1$ **to** $R_{\max}$ **do**
    Instantiate $\mathcal{A}_{II}$: $T_{\max} = II$, $minLatBetweenPEs = 0$, $maxHops \ge X + Y$
    $k \leftarrow 0$;
    $delay[0] \leftarrow 0$;
    $success \leftarrow$ **true**
    for *each node* $v \in V$ *in ASAP order* **do**
      if $\exists u \in \mathcal{U}.Group(v)$ *already in VPE* $k' \neq k$ **then** $success \leftarrow$ **false**; **break** ▷ recurrence group split: escalate II
      $(placed, hops) \leftarrow$ PlaceAndRoute$(v, \mathcal{A}_{II}, k \bmod II)$ ▷ attempt placement; returns status and hop count
      $pathDelay \leftarrow delay[k] + hops \times d_{\text{hop}} + \delta(v)$ ▷ total combinational path: accumulated + routing + op
      if *not placed* **or** $pathDelay > T_{\text{clk}}$ **then**
        if *placed* **then** Undo placement of v ▷ routable but timing violated by routing delay
        $k \leftarrow k + 1$;
        $delay[k] \leftarrow 0$ ▷ register output of current VPE, start new VPE
        $(placed, hops) \leftarrow$ PlaceAndRoute$(v, \mathcal{A}_{II}, k \bmod II)$
        if *not placed* **then** $success \leftarrow$ **false**; **break**
        $pathDelay \leftarrow hops \times d_{\text{hop}} + \delta(v)$
      $vpe[v] \leftarrow k$;
      $delay[k] \leftarrow pathDelay$
    if *success* **then** $L \leftarrow k + 1$; **return** $\mathcal{M}, II, L$
  $II \leftarrow II + 1$ ▷ escalate II: more time-slots to accommodate VPEs
return *Failure*

3.3 Virtual PE Formation

Rather than partitioning the DFG into VPEs ahead of time, *COMPOSE* forms VPEs incrementally during the physical mapping stage (Phase 3 of Alg. 2). The mapper processes nodes in ASAP order and greedily extends the current VPE k by attempting to place each node onto the CGRA at time-slot k mod II. After placement, the total combinational path delay is evaluated as $delay[k] + hops \times d_{\text{hop}} + \delta(v)$, where d_{hop} is the per-hop interconnect delay obtained from post-layout PnR data, and $hops$ is the physical routing distance returned by the placer. If the accumulated path delay remains within T_{clk}, the node is committed to the current VPE. If the delay exceeds T_{clk}, or if placement fails due to routing congestion, the mapper registers the output of the current VPE, increments k, and initiates a new VPE—effectively inserting a pipeline register boundary. The node is then re-placed in the new VPE. A critical constraint enforces recurrence correctness: if a node belongs to a recurrence group that has already been partially mapped to a different VPE, the mapper immediately declares the current II infeasible and escalates, since splitting a recurrence group across VPEs would violate the single-cycle recurrence requirement.

Upon successful placement of all nodes, the pipeline latency is $L = k + 1$ stages. If mapping fails at the current II, the framework increments II, which provides additional time-slots and PE resources, and retries. This produces a set of valid mapping points across increasing II values, enabling exploration of the throughput-latency Pareto frontier.

3.4 Microarchitectural support for *COMPOSE*

To fully realize the dynamic construction of VPEs generated by the *COMPOSE* framework, microarchitectural support is required at both the interconnect and PE datapath levels. The framework

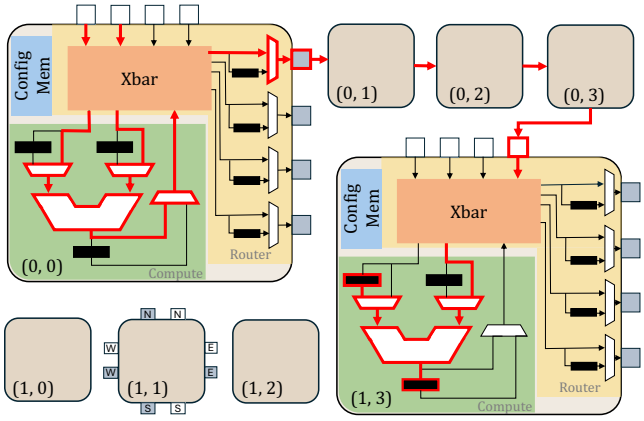


Figure 7: Spatial composition of 2 PEs across 4 hops into 1 VPE. The interconnect microarchitecture enables combinational traversal through intermediate PEs, while ALU input (*I1*,*I2*) and output (*RES*) bypass multiplexers facilitate the chaining of multiple operations. Standard registers, indicated in black, are bypassed to ensure single cycle propagation.

is parametrically adaptable to various NoC routing microarchitectures, accommodating fabrics that employ either standard single-cycle single-hop or single-cycle multi-hop routing. Because these flexible routing capabilities are already prevalent in many established academic and commercial CGRAs [10, 15, 20, 21, 32], *COMPOSE* is broadly applicable across existing chips. However, while general CGRAs support multi-hop routing, they do not natively support unlatched combinational chaining. Therefore, explicit modifications must be added to the PE datapath. Specifically, executing dependent producer-consumer pairs within the same clock cycle requires the addition of two bypass multiplexers at the input stage of each ALU. These multiplexers enable the functional unit to dynamically select between the latched data from the localized register file and the direct, unlatched operand forwarded combinationally from an upstream producer (as shown in Fig. 7). For the 4×4 Generic CGRA characterized in Section 2, integrating these input bypass multiplexers incurs a marginal hardware footprint, resulting in an area overhead of only 3.8% and a static power increment of 2.3% (see Section 5.4 for details). Crucially, this negligible structural overhead is substantially offset by significant reductions in overall dynamic power, as combinational chaining inherently eliminates the switching activity associated with intermediate register file read and write operations across the execution path.

4 Evaluation Methodology

We describe the experimental setup and methodology to evaluate the end-to-end framework for *COMPOSE* in this section. Our STA analysis toolflow was previously detailed in Section 2.2.

4.1 Compilation and Mapping

COMPOSE uses the Morpher toolchain [25] which includes dataflow graph generation, mapping, and a cycle accurate simulator written in C++. *COMPOSE* operates in the mapping stage of the Morpher toolchain. It generates static configurations for a given target architecture, clock frequency, and operation timings. The per-operation and per-hop delays are post-PnR STA values with SPEF parasitics from the taped-out chip, signed off at the Slow-Slow corner with a 5% margin. Per-hop delay does not accumulate with hop count, as each intermediate bypass PE re-drives the signal to full logic level through its crossbar. We verified this on a chain of 8 PEs and observed no degradation across hops. The mapping process of the kernels typically takes a few minutes based on the size of the DFG. Since scheduling is static, the performance is deterministic and known at compile time. *COMPOSE*'s mapping depends on the targeted cycle time, so, we evaluate T_{clk} from 100MHz to 1GHz. This range is representative of existing edge CGRA systems that operate at relatively modest frequencies, e.g. SNAFU at ~50MHz [20] and Amber [10] operates at ~500MHz. These lower clock frequencies suggest the presence of mappings with non-trivial timing slack which our approach seeks to exploit. To assess scalability, we also evaluate the framework on a larger 8×8 CGRA.

4.2 Baselines

We evaluate *COMPOSE* against a Generic CGRA baseline and two intermediate variants, each refining the previous approach. The **Generic CGRA**, modeled after HyCube [25, 33], uses simulated annealing (SA) based modulo scheduling [39], where one instruction executes per PE per cycle. We also include a CGRA-Express-like baseline that introduces compile-time operation fusion via a bypass network, following the approach of CGRA Express [36]. Fusion is restricted to neighboring PEs only and does not account for recurrence dependencies. ***COMPOSE* (Pre-Map)** introduces time-driven decomposition by partitioning the DFG based on the target clock period and mapping each partition independently with SA. Since partitioning precedes mapping, feasibility is not guaranteed, leading to fragmentation and poor utilization. ***COMPOSE* (In-Map)** interleaves partitioning and mapping greedily, improving flexibility, but does not account for recurrence dependencies or directly optimize II. Finally, *COMPOSE* extends In-Map with dynamic, recurrence-aware partitioning and mapping, incorporating loop-carried dependencies to more effectively reduce II by prioritizing full loop-carried paths within a cycle where feasible.

4.3 Kernels Used

Category	Kernel	Description	No. of nodes		Recur. length	
			u1	u4	u1	u4
Loop-carried path	dither	image dithering algo.	28	64	6	22
	llist	linked-list search	19	55	6	15
	fft	fast fourier transform	67	227	4	4
	susan	image smoothing algo.	33	78	4	6
	bfs	graph bfs	34	136	6	18
	viterbi	viterbi decoding	38	76	4	4
Bitwise-heavy	tinydes	des encryption	23	52	4	3
	popcount	population count	35	113	4	3
	crc32	32-bit crc	61	211	24	90
	aes	aes encryption	171	591	10	42
Linear algebra & AI	gemm	dense matrix mult.	26	60	4	3
	conv2d	2d convolution	39	91	4	3
	spmspm	sparse-sparse mult.	28	71	4	4
	sddmm	sampled dense mult.	28	71	4	5

Table 3: Kernels used for evaluation and their descriptions.

COMPOSE is evaluated across 14 kernels, categorized into three distinct classes to demonstrate its versatility. (1) Kernels with important inter-iteration dependencies include dither, llist, fft, susan, bfs, and viterbi. These applications possess **long loop-carried paths** that typically dictate the initiation interval, providing a rigorous test for the efficiency of the underlying mapping framework. (2) **Bitwise-heavy** kernels comprise of tinydes, popcount, aes, and crc32. Because bitwise operations have low latency, they often result in significant slack wastage. (3) **Linear Algebra and AI** kernels such as gemm, conv2d, spmspm, and sddmm represent the highly regular, compute-intensive workloads with independent loops prevalent in modern AI workloads. The kernels and their characteristics are summarized in Table 3.

5 Evaluation Results

5.1 *COMPOSE* achieves near-optimal performance & energy efficiency

Performance: Fig. 8 compares normalized cycle counts across Generic CGRA, Pre-Map, In-Map, and *COMPOSE* against the minimum achievable bound. *COMPOSE* consistently approaches the theoretical lower bound, outperforming the baseline and variants by integrating timing-aware composition with recurrence-aware mapping. On average, *COMPOSE* reduces cycles by 2.3x over Generic CGRA, 1.6x over CGRA-Express-like, 1.7x over Pre-Map and 1.4x over In-Map, with gains mostly pronounced in workloads where either recurrence constraints or slack underutilization dominate. Against the theoretical minimum ($nodes/PE_{count}$), which no schedule can beat, *COMPOSE* lands within 6.8% on average.

Pre-Map introduces timing-driven partitioning but often suffers from infeasible partitions that must be split during mapping, leading to fragmentation and sometimes even degraded cycle counts. In-Map improves feasibility by composing during mapping, but without recurrence awareness, it fails to optimally reduce initiation intervals. *COMPOSE* unifies both aspects, enabling physically realizable compositions while explicitly co-locating recurrence paths. This is further reflected in higher PE utilization (Fig. 10), where *COMPOSE* consistently achieves better resource utilization than all baselines, including CGRA-Express-like baseline, since longer cross-hop chains complete more operations per active cycle.

The impact is most visible in loop-carried workloads such as bfs and dither, where *COMPOSE* reduces cycle count by collapsing recurrence paths into fewer pipeline stages. In contrast, Generic CGRA is limited by recurrence-induced serialization, executing dependent operations across multiple cycles despite available spatial resources. CGRA-Express-like baseline keeps input-to-output latency low because its fusion is inherently short, but on recurrence-bound kernels like dither, these short chains cannot collapse the loop-carried path, so it sacrifices the initiation interval and throughput that *COMPOSE* achieves. Pre-Map and In-Map also fall short due to their inability to consistently co-locate recurrence-dependent operations, leaving a gap to the minimum bound. In bitwise-heavy workloads such as crc32, *COMPOSE* achieves significant gains by exploiting slack and packing multiple low-latency operations within a single cycle, while Pre-Map and In-Map only partially utilize this due to fragmentation or lack of global timing awareness. In contrast, workloads like gemm show minimal improvement across all methods, including *COMPOSE*, as execution is constrained by available parallelism and PE resources. ***COMPOSE* improves performance by reducing cycle count through timing-aware composition and explicit handling of recurrence dependencies.**

Energy Efficiency: Fig. 9 shows the normalized Energy-Delay Product (EDP) across all variants. *COMPOSE* consistently achieves the lowest EDP, with improvements of 6.3x over Generic CGRA, 2.9x over CGRA-Express-like, 3x over Pre-Map and 2.1x over In-Map. CGRA-Express-like baseline registers most intermediate values because its chains terminate early, and these extra register writes consume the switching energy that *COMPOSE* avoids by forwarding values within longer composed chains. While Pre-Map and In-Map reduce delay, their energy savings are limited due to additional register activity introduced because of poor partitioning and mapping. *COMPOSE*, in contrast, reduces both delay and switching activity, leading to more consistent EDP improvements across workloads.

Fig. 11 shows the reduction in intermediate register usage. *COMPOSE* reduces registering by 45% compared to Generic CGRA, 29% compared to CGRA-Express-like and 31% compared to PreMap by forwarding values within composed execution chains instead of writing them back at every PE boundary. Pre-Map and In-Map still insert register boundaries whenever compositions cannot be maintained across cycles, which increases the number of intermediate writes. This difference is more pronounced in workloads with long dependent chains, where *COMPOSE* can keep values within the combinational datapath across multiple operations. ***COMPOSE* reduces EDP by minimizing intermediate register accesses and unnecessary data movement.**

Real-time Performance: Fig 9 also shows the input-to-output latency across benchmarks. *COMPOSE* improves initiation interval and overall throughput, but this comes with a modest increase in pipeline depth, leading to slightly higher input-to-output latency compared to In-Map. In most workloads, this increase is limited to one additional pipeline stage, while in cases such as fft, the increase is more noticeable due to deeper dependency chains. In contrast, In-Map achieves lower input-to-output latency by maintaining a shallower pipeline, but at the cost of a higher initiation interval and reduced throughput. Pre-Map exhibits higher latency due to its coarse partitioning and is not competitive in this trade-off. CGRA-Express-like baseline fuses only adjacent operations, so on kernels with longer chains it forms more pipeline stages than *COMPOSE*, giving higher input-to-output latency, while still failing to reduce the initiation interval on recurrence-bound kernels like dither. **This highlights a clear trade-off between throughput and latency. *COMPOSE* prioritizes throughput by reducing initiation interval through composition, while In-Map prioritizes latency by limiting pipeline depth.**

5.2 Ablation Study

Interconnect Model: Fig. 12 compares *COMPOSE* across two fabrics: (i) single-cycle single-hop routing and (ii) single-cycle multi-hop routing, reporting performance (normalized cycle count) and the number of VPEs formed. In the multi-hop design, *COMPOSE* incorporates STA-derived router-to-router delay (crossbar delay) and

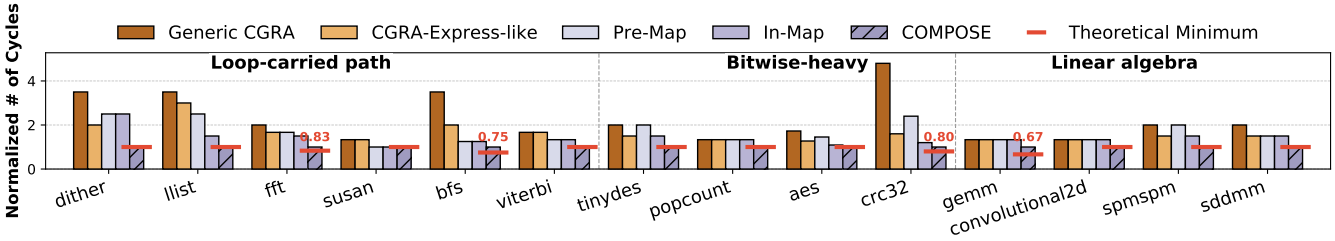


Figure 8: Normalized cycle count across workloads comparing baselines and *COMPOSE* against the theoretical minimum.

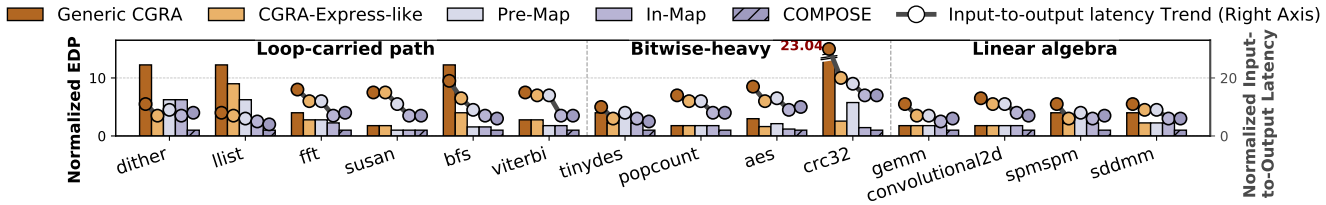


Figure 9: Normalized EDP and input-to-output latency across workloads for all baselines and *COMPOSE*.

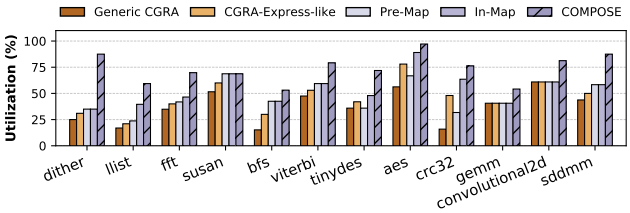


Figure 10: Improved PE utilization with *COMPOSE*.

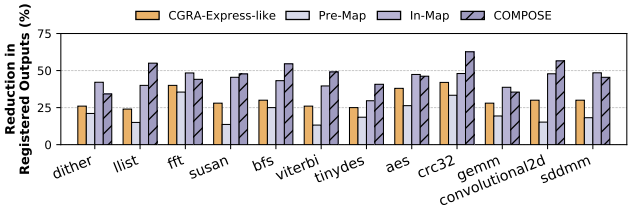


Figure 11: Reduction in intermediate register writes compared to Generic CGRA.

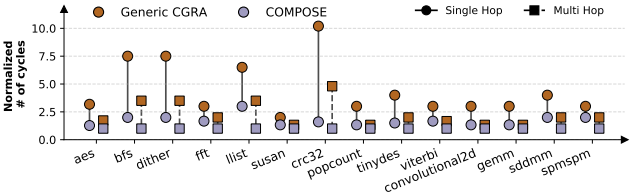


Figure 12: Cycle counts normalized to *COMPOSE* multi-hop across benchmarks. *COMPOSE* consistently achieves lower latency than Generic CGRA under both single-hop and multi-hop router communication.

scales it with hop count to determine whether a chain fits within a cycle. This directly affects VPE formation, allowing a single VPE to span multiple hops when timing permits. As a result, fewer but larger VPEs are formed, which improves cycle count by increasing the amount of work completed within each cycle.

In contrast, single-hop routing restricts VPE formation to adjacent PEs, forcing earlier VPE termination when dependencies span longer distances. This results in a higher number of smaller VPEs and increased pipeline boundaries, which leads to higher cycle counts. The difference is more pronounced in workloads such as bfs and fft, where dependencies are distributed across the fabric, while workloads like gemm show smaller differences due to localized mappings. **Overall, the reduction in VPE count in the multi-hop case directly correlates with improved performance, highlighting that flexible routing enables more effective composition.**

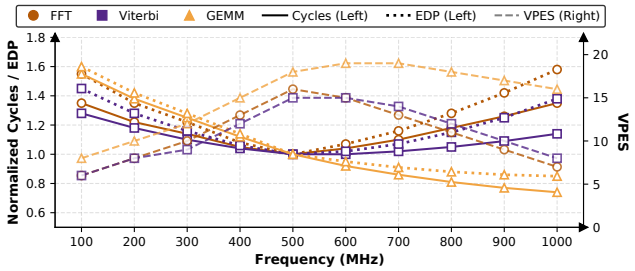


Figure 13: Impact of operating frequency on execution time, EDP, and VPE count across workloads.

Frequency Variation: Fig. 13 shows the impact of operating frequency on normalized cycles, EDP, and VPE count for fft, viterbi, and gemm, which represent three classes of workloads. For fft and viterbi, the optimal operating point occurs around 500 MHz, where *COMPOSE* achieves the best trade-off between composition and timing. At this frequency, sufficient slack exists to form larger VPEs, enabling the mapper to reach the achievable II. As frequency increases, the tighter T_{clk} restricts VPE formation, reducing composition opportunities and increasing VPE count. This shifts the bottleneck to recurrence constraints in fft and limits further gains in

viterbi. In contrast, gemm continues to benefit at higher frequencies since it is primarily resource-bound and less dependent on composition. **Thus, the optimal frequency is not the highest clock frequency, but the point where *COMPOSE* maximizes VPE size while avoiding recurrence-limited execution, enabling peak performance at moderate frequencies.**

5.3 *COMPOSE* scales to large CGRAs

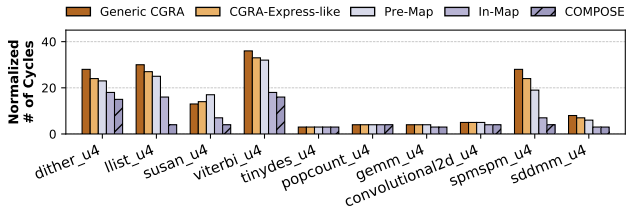


Figure 14: Scalability on 8×8 CGRA for large DFGs; cycles normalized to *COMPOSE* show consistent improvements.

Fig. 14 evaluates *COMPOSE* on an 8×8 CGRA, showing that its benefits persist as the fabric size increases. *COMPOSE* continues to achieve lower cycle counts and improved EDP compared to all baselines, while maintaining efficient VPE formation across a larger spatial domain. The larger fabric provides more placement flexibility, enabling the mapper to form VPEs with longer chains without being constrained by local resource availability. CGRA-Express-like baseline benefits less from this, as neighbor-only fusion cannot reach across the fabric to form longer chains. At the same time, *COMPOSE* effectively manages the increased routing complexity by incorporating hop-aware timing into VPE formation, ensuring that compositions remain timing-valid. ***COMPOSE* scales with CGRA size by converting increased spatial flexibility into larger VPEs and improved execution efficiency.**

5.4 Overhead of *COMPOSE*

COMPOSE introduces modest hardware overheads of 3.8% in area and 2.3% in power, arising from additional bypassing support in the PE datapath, specifically two input multiplexers and one predication multiplexer per PE, while output bypass paths already exist in generic CGRA designs. *COMPOSE* increases compilation time by approximately 1.6×, as VPE boundaries must be determined dynamically based on timing rather than fixed pipeline stages. **However, given the small hardware overhead and consistent performance gains, this trade-off is practical.**

5.5 *COMPOSE* Generalizes Across Datatypes

Evaluation so far uses the integer datapath of our taped-out chip. To show *COMPOSE* is not tied to a datatype, we synthesized and placed-and-routed an FP16 ALU and extracted its per-operation delays through the same STA flow as Section 2.2 (post-PnR, not silicon-verified). Since *COMPOSE* consumes per-operation timing as an input, the framework is unchanged: the FP16 delays simply replace the integer $\delta(v)$ values during VPE formation.

Fig. 15 reports FP16 cycle counts. *COMPOSE* retains up to a 1.7× cycle reduction over Generic CGRA (on fft), smaller than the integer



Figure 15: FP16 cycle counts normalized to *COMPOSE*, with largest gains on recurrence-heavy kernels.

case because wider arithmetic has longer critical paths and leaves less slack to compose, most visible on the recurrence-heavy fft and susan. **Datatype only shifts the per-operation delays fed to VPE formation, so wider datapaths reduce composition opportunity without changing the framework or diminishing *COMPOSE*'s advantage over the baseline.**

6 Other Related Works

Prior work can be broadly categorized based on the level at which they improve execution: (i) composability at the level of cores or accelerators, (ii) enhancements to PEs, and (iii) micro-architectural fusion and datapath pipelining. We discuss them in the context of composability and accelerating inter-iteration dependencies.

Core-level composability. Prior work has explored composability at coarser granularities, particularly at the level of processor cores or accelerator units. Stitch [46], for example, proposes a many-core architecture where instruction set extension units are combined to form larger virtual accelerators. Similarly, [9] enables runtime aggregation of small cores into more powerful logical processors to match application parallelism. While these approaches demonstrate the benefits of composability via compiler-scheduled NoCs and runtime reconfiguration, they operate at a coarse granularity and do not expose mechanisms for composing fine-grained compute elements as in a CGRA datapath.

CGRAs with enhanced PEs. Another approach to improve CGRA efficiency is increasing the functional richness of individual PEs while preserving a fixed abstraction. Plaid [28] introduces motif-based execution, mapping recurring communication patterns onto specialized units with three ALUs and a local router. Unlike Plaid's fixed configuration, *COMPOSE* supports flexible virtual PE boundaries, enabling a variable number of ALUs. RF-CGRA [52] enhances interconnects for multi-cycle dependencies using register chains, whereas *COMPOSE* improves efficiency for short logic paths by fusing operations and eliminating intermediate registers. FPCA [14] explores dynamic runtime composition for parallel workloads, which is complementary to *COMPOSE*'s compile-time approach. NUPEA [19] also starts from recurrence analysis but targets data movement, steering latency-critical loads into near-memory domains during place-and-route. *COMPOSE* instead uses recurrence information to drive composition; the two are complementary and could be combined.

Micro-architectural fusion and datapath pipelining. Another line of work improves performance through micro-architectural optimizations and scheduling. Capstone [49] extends Cascade [31] with a compiler-driven energy model to enforce power constraints. FLAME [38] decomposes long-latency operations into multi-cycle

execution for high-frequency operation. In contrast, *COMPOSE* targets lower-frequency regimes, achieving comparable or better performance by fusing operations into a combinational datapath. More broadly, HLS harvests slack at synthesis time via cycle-time-aware scheduling and operation chaining [40, 43], with FPGA-targeted flows relying on estimated delay models [51]; unlike a CGRA mapper, none expose measured post-layout STA delays to spatial placement on a fixed fabric or resolve the loop-carried recurrences that bound throughput. Beyond CGRAs, DPU-v2 [42] accelerates irregular DAGs by mapping them into tree-like structures aligned with its datapath.

Prior works optimize hardware and software in isolation, neglecting significant timing slack from CGRA operation variability. ***COMPOSE* bridges this divide through hardware-software co-design, exposing circuit timing directly to the compiler.**

7 Discussion

Understanding *COMPOSE* through VLIW lens: While both *COMPOSE* and VLIW leverage instruction-level parallelism (ILP) [16] by grouping operations into execution packets, they differ in managing temporal constraints. In traditional VLIW, the compiler packs independent operations that execute in parallel across multiple functional units in lock-step. This synchronous execution often forces faster units to wait for the slowest operation in the bundle, creating significant timing slack. *COMPOSE* extends this concept through timing-driven spatial composition, allowing dependent producer-to-consumer operations to execute back-to-back within the same cycle. By utilizing STA to harvest slack from low-latency primitives and interconnect hops, *COMPOSE* reduces slack within the lock-step itself. This enables the framework to collapse multi-cycle recurrence paths and resolve complex dependencies in one cycle, avoiding the intermediate register file write-backs that standard VLIW architectures require for dependent chains.

Understanding *COMPOSE*'s Chaining vs Cray-1's Vector Chaining [41]: While *COMPOSE* and the Cray-1 both utilize chaining, they differ fundamentally in their scheduling mechanisms and architectural intent. Cray-1's vector chaining is a runtime hardware process governed by a narrow chain slot time. If the dependent instruction is not ready to start at the exact moment the first result element exits the pipeline, the hardware misses the slot and forces the second instruction to wait for the entire vector to finish writing. In contrast, *COMPOSE*'s compilation strategy is entirely static and pre-calculates valid chaining opportunities before execution. While the Cray-1 chains operations across discrete clocked pipeline stages, *COMPOSE* chains operations within a single clock cycle.

8 Conclusion

This paper demonstrates that the rigid abstraction of fixed-cycle execution in CGRAs leaves significant performance gains and energy savings on the table. *COMPOSE* addresses this by exposing circuit-level timing to the compiler, allowing composable execution through dynamically formed virtual PEs. This is especially beneficial for recurrence-bound loops where registered stages on such paths directly increase recurrence latency and can raise II. By collapsing recurrence-critical producer-consumer chains into fewer registered stages, *COMPOSE* improves utilization, performance, energy, and resource efficiency, highlighting the value of tighter hardware-software co-design.

References

[1] [n. d.]. SN40L RDU AI Chip. https://sambanova.ai/products/sn40l-rdu-ai-chip. Accessed: 2025-08-14.

[2] Giovanni Ansaloni, Paolo Bonzini, and Laura Pozzi. 2008. Design and Architectural Exploration of Expression-Grained Reconfigurable Arrays. In *2008 Symposium on Application Specific Processors*. 26–33. https://doi.org/10.1109/SASP.2008.4570782

[3] Giovanni Ansaloni, Paolo Bonzini, and Laura Pozzi. 2009. Heterogeneous coarse-grained processing elements: a template architecture for embedded processing acceleration. In *Proceedings of the Conference on Design, Automation and Test in Europe* (Nice, France) *(DATE '09)*. European Design and Automation Association, Leuven, BEL, 542–547.

[4] Giovanni Ansaloni, Paolo Bonzini, and Laura Pozzi. 2011. EGRA: A Coarse Grained Reconfigurable Architectural Template. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems* 19, 6 (2011), 1062–1074. https://doi.org/10.1109/TVLSI.2010.2044667

[5] Giovanni Ansaloni, Laura Pozzi, Kazuyuki Tanimura, and Nikil Dutt. 2011. Slack-aware scheduling on Coarse Grained Reconfigurable Arrays. In *2011 Design, Automation & Test in Europe*. 1–4. https://doi.org/10.1109/DATE.2011.5763323

[6] Zhenyu Bai, Pranav Dangi, Rohan Juneja, Zhaoying Li, Zhanglu Yan, Huiying Lan, and Tulika Mitra. 2025. A Data-Driven Dynamic Execution Orchestration Architecture. In *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Pittsburgh, PA, USA) *(ASPLOS '26)*. Association for Computing Machinery, New York, NY, USA, 1–19. https://doi.org/10.1145/3760250.3762226

[7] J. Bhasker and Rakesh Chadha. 2009. *Static Timing Analysis for Nanometer Designs: A Practical Approach* (1st ed.). Springer Publishing Company, Incorporated.

[8] David Blaauw, Kaviraj Chopra, Ashish Srivastava, and Lou Scheffer. 2008. Statistical Timing Analysis: From Basic Principles to State of the Art. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 27, 4 (2008), 589–607. https://doi.org/10.1109/TCAD.2007.907047

[9] Doug Burger, Stephen W. Keckler, and Simha Sethumadhavan. 2009. *Composable Multicore Chips*. Springer US, Boston, MA, 73–109. https://doi.org/10.1007/978-1-4419-0263-4_3

[10] Alex Carsello and et el. [n. d.]. Amber: A 367 GOPS, 538 GOPS/W 16nm SoC with a Coarse-Grained Reconfigurable Array for Flexible Acceleration of Dense Linear Algebra. In *VLSI'22*. 70–71. https://doi.org/10.1109/VLSITechnologyandCir46769.2022.9830509

[11] N. Clark, J. Blome, M. Chu, S. Mahlke, S. Biles, and K. Flautner. 2005. An architecture framework for transparent instruction set customization in embedded processors. In *32nd International Symposium on Computer Architecture (ISCA'05)*. 272–283. https://doi.org/10.1109/ISCA.2005.9

[12] Nathan Clark, Amir Hormati, Scott Mahlke, and Sami Yehia. 2006. Scalable subgraph mapping for acyclic computation accelerators. In *Proceedings of the 2006 International Conference on Compilers, Architecture and Synthesis for Embedded Systems* (Seoul, Korea) *(CASES '06)*. Association for Computing Machinery, New York, NY, USA, 147–157. https://doi.org/10.1145/1176760.1176779

[13] N. Clark, M. Kudlur, Hyunchul Park, S. Mahlke, and K. Flautner. 2004. Application-Specific Processing on a General-Purpose Core via Transparent Instruction Set Customization. In *37th International Symposium on Microarchitecture (MICRO-37'04)*. 30–40. https://doi.org/10.1109/MICRO.2004.5

[14] Jason Cong, Hui Huang, Chiyuan Ma, Bingjun Xiao, and Peipei Zhou. 2014. A Fully Pipelined and Dynamically Composable Architecture of CGRA. In *2014 IEEE 22nd Annual International Symposium on Field-Programmable Custom Computing Machines*. 9–16. https://doi.org/10.1109/FCCM.2014.12

[15] Efficient Computer. [n. d.]. Electron E1. https://www.efficient.computer/electron-e1 Accessed: 2026-04-06.

[16] Joseph A. Fisher. 1983. Very Long Instruction Word architectures and the ELI-512. *SIGARCH Comput. Archit. News* 11, 3 (June 1983), 140–150. https://doi.org/10.1145/1067651.801649

[17] Kermin E Fleming and et al. 2020. Processors, methods, and systems with a configurable spatial accelerator. US Patent 10,558,575.

[18] Taro Fujii and et al. [n. d.]. New Generation Dynamically Reconfigurable Processor Technology for Accelerating Embedded AI Applications. In *VLSI'18*. 41–42. https://doi.org/10.1109/VLSIC.2018.8502438

[19] Souradip Ghosh, Graham Gobieski, Keyi Zhang, Brandon Lucia, Nathan Beckmann, and Tony Nowatzki. 2025. NUPEA: Optimizing Critical Loads on Spatial Dataflow Architectures via Non-Uniform Processing-Element Access. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 1627–1640. https://doi.org/10.1145/3695053.3731061

[20] Graham Gobieski, Ahmet Oguz Atli, Kenneth Mai, Brandon Lucia, and Nathan Beckmann. [n. d.]. Snafu: An Ultra-Low-Power, Energy-Minimal CGRA-Generation Framework and Architecture. In *ISCA'21*. 1027–1040. https://doi.org/10.1109/ISCA52012.2021.00084

[21] Graham Gobieski, Souradip Ghosh, Marijn Heule, Todd Mowry, Tony Nowatzki, Nathan Beckmann, and Brandon Lucia. 2023. RipTide: A Programmable, Energy-Minimal Dataflow Compiler and Architecture. In *Proceedings of the 55th Annual IEEE/ACM International Symposium on Microarchitecture* (Chicago, Illinois, USA) *(MICRO '22)*. IEEE Press, 546–564. https://doi.org/10.1109/MICRO56248.2022.00046

[22] Shantanu Gupta, Shuguang Feng, Amin Ansari, Scott Mahlke, and David August. 2011. Bundled execution of recurring traces for energy-efficient general purpose processing. In *Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture* (Porto Alegre, Brazil) *(MICRO-44)*. Association for Computing Machinery, New York, NY, USA, 12–23. https://doi.org/10.1145/2155620.2155623

[23] David Money Harris, Ron Ho, Gu-Yeon Wei, and Mark Horowitz. 1998. The Fanout-of-4 Inverter Delay Metric. https://api.semanticscholar.org/CorpusID:9167634

[24] Natalie Enright Jerger, Tushar Krishna, and Li-Shiuan Peh. 2017. *On-Chip Networks: Second Edition* (2nd ed.). Morgan & Claypool Publishers.

[25] Rohan Juneja, Pranav Dangi, Thilini Kaushalya Bandara, Zhaoying Li, Dhananjaya Wijerathne, Li-Shiuan Peh, and Tulika Mitra. 2025. Building an Open CGRA Ecosystem for Agile Innovation. arXiv:2508.19090 [cs.AR] https://arxiv.org/abs/2508.19090

[26] Rohan Juneja, Pranav Dangi, Thilini Kaushalya Bandara, Tulika Mitra, and Li-Shiuan Peh. 2025. Nexus Machine: An Energy-Efficient Active Message Inspired Reconfigurable Architecture. In *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25)*. Association for Computing Machinery, New York, NY, USA, 1221–1235. https://doi.org/10.1145/3725843.3756091

[27] Changmoo Kim, Mookyoung Chung, Yeongon Cho, Mario Konijnenburg, Soojung Ryu, and Jeongwook Kim. 2012. ULP-SRP: Ultra low power Samsung Reconfigurable Processor for biomedical applications. In *2012 International Conference on Field-Programmable Technology*. 329–334. https://doi.org/10.1109/FPT.2012.6412157

[28] Zhaoying Li, Pranav Dangi, Chenyang Yin, Thilini Kaushalya Bandara, Rohan Juneja, Cheng Tan, Zhenyu Bai, and Tulika Mitra. 2025. Enhancing CGRA Efficiency Through Aligned Compute and Communication Provisioning. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 410–425. https://doi.org/10.1145/3669940.3707230

[29] Bingfeng Mei, Serge Vernalde, Diederik Verkest, Haris Man, and Rudy Lauwereins. [n. d.]. ADRES: An Architecture with Tightly Coupled VLIW Processor and Coarse-Grained Reconfigurable Matrix. In *FPL'03*. https://doi.org/10.1007/978-3-540-45234-8_7

[30] Jackson Melchert, Kathleen Feng, Caleb Donovick, Ross Daly, Ritvik Sharma, Clark Barrett, Mark A. Horowitz, Pat Hanrahan, and Priyanka Raina. 2023. APEX: A Framework for Automated Processing Element Design Space Exploration using Frequent Subgraph Analysis. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3* (Vancouver, BC, Canada) *(ASPLOS 2023)*. Association for Computing Machinery, New York, NY, USA, 33–45. https://doi.org/10.1145/3582016.3582070

[31] Jackson Melchert, Yuchen Mei, Kalhan Koul, Qiaoyi Liu, Mark Horowitz, and Priyanka Raina. 2024. Cascade: An Application Pipelining Toolkit for Coarse-Grained Reconfigurable Arrays. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 43, 10 (2024), 3055–3067. https://doi.org/10.1109/TCAD.2024.3390542

[32] V. P. Nambiar, Y. S. Chong, T. K. Bandara, D. Wijerathne, Z. Li, R. Juneja, L.-S. Peh, T. Mitra, and A. T. Do. 2024. A 360 GOPS/W CGRA in a RISC-V SoC with Multi-Hop Routers and Idle-State Instructions for Edge Computing Applications. In *2024 21st International SoC Design Conference (ISOCC)*. 89–90. https://doi.org/10.1109/ISOCC62682.2024.10762131

[33] Vishnu P. Nambiar, Yi Sheng Chong, Thilini Kaushalya Bandara, Dhananjaya Wijerathne, Zhaoying Li, Rohan Juneja, Li-Shiuan Peh, Tulika Mitra, and Anh Tuan Do. 2024. PACE: A Scalable and Energy Efficient CGRA in a RISC-V SoC for Edge Computing Applications. In *2024 IEEE Hot Chips 36 Symposium (HCS)*. 1–1. https://doi.org/10.1109/HCS61935.2024.10665106

[34] Tony Nowatzki, Vinay Gangadhar, Newsha Ardalani, and Karthikeyan Sankaralingam. 2017. Stream-Dataflow Acceleration. *SIGARCH Comput. Archit. News* 45, 2 (June 2017), 416–429. https://doi.org/10.1145/3140659.3080255

[35] Taewook Oh, Bernhard Egger, Hyunchul Park, and Scott Mahlke. 2009. Recurrence cycle aware modulo scheduling for coarse-grained reconfigurable architectures. In *Proceedings of the 2009 ACM SIGPLAN/SIGBED Conference on Languages, Compilers, and Tools for Embedded Systems* (Dublin, Ireland) *(LCTES '09)*. Association for Computing Machinery, New York, NY, USA, 21–30. https://doi.org/10.1145/1542452.1542456

[36] Yongjun Park, Hyunchul Park, and Scott Mahlke. 2009. CGRA express: accelerating execution using dynamic operation fusion. In *Proceedings of the 2009 International Conference on Compilers, Architecture, and Synthesis for Embedded Systems* (Grenoble, France) *(CASES '09)*. Association for Computing Machinery, New York, NY, USA, 271–280. https://doi.org/10.1145/1629395.1629433

[37] Raghu Prabhakar and Sumti Jairath. 2021. SambaNova SN10 RDU:Accelerating Software 2.0 with Dataflow. In *2021 IEEE Hot Chips 33 Symposium (HCS)*. 1–37. https://doi.org/10.1109/HCS52781.2021.9567250

[38] Jiajun Qin, Cheng Tan, Ruihong Yin, Tianhua Xia, Sai Qian Zhang, and Bei Yu. [n. d.]. FLAME: A Framework Exploring Execution Strategies for Multi-Cycle Operations in CGRA. ([n. d.]).

[39] B. Ramakrishna Rau. 1994. Iterative modulo scheduling: an algorithm for software pipelining loops. In *Proceedings of the 27th Annual International Symposium on Microarchitecture* (San Jose, California, USA) *(MICRO 27)*. Association for Computing Machinery, New York, NY, USA, 63–74. https://doi.org/10.1145/192724.192731

[40] R. Ruiz-Sautua, M.C. Molina, J.M. Mendias, and R. Hermida. 2005. Behavioural transformation to improve circuit performance in high-level synthesis. In *Design, Automation and Test in Europe*. 1252–1257 Vol. 2. https://doi.org/10.1109/DATE.2005.81

[41] Richard M. Russell. 1978. The CRAY-1 computer system. *Commun. ACM* 21, 1 (Jan. 1978), 63–72. https://doi.org/10.1145/359327.359336

[42] Nimish Shah, Wannes Meert, and Marian Verhelst. 2022. DPU-v2: Energy-efficient execution of irregular directed acyclic graphs. In *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1288–1307. https://doi.org/10.1109/MICRO56248.2022.00090

[43] Mukund Sivaraman and Shail Aditya. 2002. Cycle-time aware architecture synthesis of custom hardware accelerators. In *Proceedings of the 2002 International Conference on Compilers, Architecture, and Synthesis for Embedded Systems* (Grenoble, France) *(CASES '02)*. Association for Computing Machinery, New York, NY, USA, 35–42. https://doi.org/10.1145/581630.581637

[44] Aaron Stillmaker and Bevan Baas. 2017. Scaling equations for the accurate prediction of CMOS device performance from 180nm to 7nm. *Integration* 58 (2017), 74–81. https://doi.org/10.1016/j.vlsi.2017.02.002

[45] Ivan Sutherland, Bob Sproull, and David Harris. 1999. *Logical effort: designing fast CMOS circuits*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA.

[46] Cheng Tan, Manupa Karunaratne, Tulika Mitra, and Li-Shiuan Peh. 2018. Stitch: fusible heterogeneous accelerators enmeshed with many-core architecture for wearables. In *Proceedings of the 45th Annual International Symposium on Computer Architecture* (Los Angeles, California) *(ISCA '18)*. IEEE Press, 575–587. https://doi.org/10.1109/ISCA.2018.00054

[47] Christopher Torng, Peitian Pan, Yanghui Ou, Cheng Tan, and Christopher Batten. 2021. Ultra-Elastic CGRAs for Irregular Loop Specialization. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 412–425. https://doi.org/10.1109/HPCA51647.2021.00042

[48] Dhananjaya Wijerathne, Zhaoying Li, Manupa Karunarathne, Anuj Pathania, and Tulika Mitra. 2019. CASCADE: High Throughput Data Streaming via Decoupled Access-Execute CGRA. *ACM Trans. Embed. Comput. Syst.* 18, 5s, Article 50 (Oct. 2019), 26 pages. https://doi.org/10.1145/3358177

[49] Sabrina Yarzada and Christopher Torng. 2026. Capstone: Power-Capped Pipelining for Coarse-Grained Reconfigurable Array Compilers. arXiv:2603.00909 [cs.AR] https://arxiv.org/abs/2603.00909

[50] Sami Yehia, Nathan Clark, Scott Mahlke, and Krisztián Flautner. 2005. Exploring the design space of LUT-based transparent accelerators. 11–21. https://doi.org/10.1145/1086297.1086301

[51] David C. Zaretsky, Gaurav Mittal, Robert P. Dick, and Prith Banerjee. 2007. Balanced Scheduling and Operation Chaining in High-Level Synthesis for FPGA Designs. In *8th International Symposium on Quality Electronic Design (ISQED'07)*. 595–601. https://doi.org/10.1109/ISQED.2007.41

[52] Rong Zhu, Bo Wang, and Dajiang Liu. 2022. RF-CGRA: A Routing-Friendly CGRA with Hierarchical Register Chains. In *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. 262–267. https://doi.org/10.23919/DATE54114.2022.9774601